

Entropy-Based Network Intrusion Detection using Markov Chains

Ignatios Mavrelis

22031

ece22031@go.uop.gr

Supervising Professor: Panagiotis Zervas

Department of Electrical and Computer Engineering
University of Peloponnese

23 September 2026

Abstract

The rapid evolution of network threats makes traditional detection methods, which rely primarily on static signatures, increasingly less effective. This report presents the design, implementation, and evaluation of a network anomaly detection system based on metadata. Instead of operating by checking for known attacks (like Deep Packet Inspection - DPI), it approaches the problem from the perspective of behavioral analysis through traffic metadata, creating a profile of normal network behavior. Using theoretical approaches from Information Theory, such as Shannon entropy, combined with first-order Markov chains ($k = 1$), the system monitors traffic flow and detects deviations without any knowledge of the payload format. Its architecture is framed by multiple sensors (Multi-Sensor Fusion) operating simultaneously, dramatically reducing errors and offering absolute explainability, unlike modern "black box" Neural Networks (*Goodfellow et al., 2016*).

Keywords: Network Intrusion Detection, Shannon Entropy, Markov Chains, Unsupervised Learning, Multi-Sensor Fusion, Cybersecurity, Data Discretization, Deterministic Algorithms

Contents

1	Introduction	3
1.1	Introduction and Problem Definition	3
1.2	Project Objectives	3
1.3	Report Structure	3
2	Theoretical Background and Literature Review	4
2.1	Discrete Source Theory and Entropy	4
2.2	Modeling Stochastic Processes	4
2.3	Literature Review and Different Approaches	4
2.4	Conclusions and State of the Art	5
2.5	Comparative Method Analysis	5
2.6	Deviation Evaluation (Cross-Entropy)	6
3	Methodology	6
3.1	Dataset Description	6
3.2	Feature Selection and Data Engineering	7
3.3	Discretization and Smoothing	7
3.4	Model Architecture and "Training" Process	8
4	Implementation	8
4.1	Development Environment	8
4.2	Multi-Sensor Architecture	9
5	Experiments and Results	9
5.1	Baseline Results	9
5.2	Graphical Evaluation	11
6	Discussion	14
6.1	Interpretation of Results	14
6.2	System Limitations	14
6.3	Additional Features and Extensions (BETA)	15
7	Conclusions	15
7.1	Project Summary	15
7.2	Future Work	15
A	Appendix A: Code Snippets	18

List of Figures

1	Drop in entropy as the model order (k) increases	11
2	Normalized Deviation of multiple sensors during attacks	12
3	Performance evaluation (ROC Curves & Confusion Matrix) of the model	13

List of Tables

1	Number of Unique States and Baseline Entropy Depending on Model Order k (Sequence Size: 488,381)	9
---	--	---

1 Introduction

1.1 Introduction and Problem Definition

In the modern era, cybersecurity is one of the most critical pillars for the operation of organizations and state infrastructures. However, traditional security methods, such as classic Firewalls and other Intrusion Prevention Systems, significantly underperform against Zero-day attacks. These systems, essentially operating as “detectors,” rely on the existence of **already known signatures**. If an attack is new and not in the database, it passes completely unnoticed.

This paper proposes the development of a system that more closely simulates the human immune system. Such a Network Behavior Anomaly Detector (NBAD) system does not need to know the profile of all threats from the beginning. Instead, it thoroughly learns what a “healthy state” means for the specific organization and raises an alarm when it detects a statistically significant deviation from normal.

The advantage of this architecture is that it avoids inspecting the user files/-data themselves (DPI). Today, approximately 90% of internet traffic is encrypted (via HTTPS/TLS), so reading the payload is practically impossible. The solution lies in the **traffic metadata**, meaning the flow characteristics (NetFlow), such as communication duration and data volume.

By using Conditional Entropy, the system monitors the stability of the flows. At rest, a network has a predictable, relatively high entropy that indicates traffic diversity (many users, different pages). But when a threat occurs, for example a Distributed Denial of Service (DDoS) attack or Port Scanning, thousands of identical packets are sent en masse. This mechanical repetition causes a violent collapse of uncertainty (making the network “predictable”) and, consequently, a measurable drop in entropy, which acts as the intrusion signal for the system.

1.2 Project Objectives

The main objective of this work is to demonstrate the efficiency of an alternative intrusion detection architecture, which differs from established mainstream methods. Given that new cyberattacks are constantly developed by malicious actors who ceaselessly evolve their techniques, it is imperative for security managers to seek innovative approaches beyond traditional tools to effectively protect their infrastructures.

At the same time, this research serves as the foundational, static part of a broader research framework. The ultimate goal is not only the theoretical design of a behavior-based detection system but the future simulation of a complete network environment where the proposed scheme will transition from static offline analysis to full operation and real-time evaluation.

1.3 Report Structure

The remainder of this report is organized as follows: Chapter 2 presents the theoretical background, analyzing the concepts of Shannon entropy and Markov chains, as well as an overview of modern approaches. Chapter 3 delves into the methodology followed, emphasizing Data Engineering and the training process. Chapter 4 describes the implementation of the Multi-Sensor Fusion architecture. Chapter 5 presents the experiments and evaluates the models’ results, while Chapter 6 provides an extensive discussion on the findings and the system’s limitations. Finally, Chapter 7 summarizes the conclusions and suggests directions for future research.

2 Theoretical Background and Literature Review

The second chapter introduces the fundamental mathematics upon which the system was built. Avoiding the complexity of Machine Learning models that lack explainability, our analysis relies on robust theoretical approaches from Probability Theory.

2.1 Discrete Source Theory and Entropy

As described by discrete source theory, every communication system can be modeled as a source emitting symbols from a finite alphabet. Measuring the uncertainty surrounding the next symbol is done using the entropy formula (Zervas, 2026). This mathematical quantity represents the **information uncertainty** (Shannon, 1948). Given a sequence of events, Conditional Entropy is defined as:

$$H(Y|X) = - \sum_{x,y} p(x,y) \log_2 p(y|x) \quad (1)$$

In the context of our network, X represents the current network behavior, while Y represents the immediate next step. This equation measures the uncertainty of the next packet given the past. As observed in previous research (Nychis *et al.*, 2008), an automated network attack creates overly repetitive behavior, as malware executes the same loop of operations. Consequently, the uncertainty of the next step is almost zeroed out, which is reflected in a dramatic drop in entropy.

2.2 Modeling Stochastic Processes

To map the sequence of packets (which constitutes a stochastic process), the mathematical model of Markov Chains was chosen, which is trained using a strictly deterministic frequency counting algorithm, offering absolute stability and repeatability in the results. The critical question during design concerned the choice of the chain's order (k).

The choice of order $k = 1$ is explicitly documented in literature for intrusion prevention systems (Ye *et al.*, 2002) and is confirmed in the project's experimental process below. If we chose a larger k (e.g., 3 or 5), the chain would attempt to memorize combinations of three or consecutive packets. Although this theoretically provides greater "memory" to the system, practically the size of the transition matrix increases exponentially, creating thousands of empty states.

This phenomenon, State-Space Explosion, causes the training entropy to collapse (since most transitions have a probability of 1 or 0). The system moves from the "learning" phase to the "memorization" phase (Overfitting). During evaluation on real data, any normal but unknown variation in traffic is misconstrued as an attack, leading to a dramatic increase in False Positives. Therefore, the 1st order Markov Chain constitutes the golden ratio between performance and generalization capability.

2.3 Literature Review and Different Approaches

In terms of stochastic model comparison, Ye *et al.* (2002) studied the difference between first-order Markov chains and higher-order stochastic models for analyz-

ing sequences of audit data. Their research demonstrated that a first-order model represents the ideal intersection between computational speed and accuracy, outperforming more complex models in real-time conditions (Ye *et al.*, 2002). Expanding on the same thought, Ye, Zhang, and Borror (2004) evaluated the robustness of these Markov models, proving that although they work flawlessly on “clean” data, their accuracy drops rapidly as network noise increases, highlighting the need for careful traffic preprocessing (Ye *et al.*, 2004).

In the field of Information Theory, Lee and Xiang (2001) introduced advanced metrics (such as Conditional Entropy and Information Gain) for automating model creation. Their method eliminates the need for trial-and-error testing by mathematically calculating the optimal size of the traffic observation time window (Lee and Xiang, 2001). Meanwhile, Nychis *et al.* (2008) empirically evaluated entropy-based detection, concluding that IP address and port distributions are redundant if not overly correlated, and that modern systems must examine behavioral features (such as In/Out-degree dispersion) to detect complex scanning and P2P network attacks (Nychis *et al.*, 2008).

A different approach was the use of compression algorithms for anomaly detection. Callegari *et al.* (2009) applied compression algorithms (Huffman, LZW, Dynamic Markov Coding) based on the logic that entropy determines the compression limit. Thus, traffic is considered anomalous if it deviates from the expected compression “dictionary” of normal TCP traffic (Callegari *et al.*, 2009). More recently, Ting *et al.* (2018) dramatically evolved this idea by introducing the “Slice Compression” (SCADe) method via NCD (Normalized Compression Distance). Their system can identify local anomalies without any prior knowledge of protocol characteristics (Ting *et al.*, 2018).

Finally, as summarized in the extensive literature review by Ahmed, Mahmood, and Hu (2016), the scientific community has now categorized techniques into statistical, classification, and clustering, clearly demonstrating the shift towards unsupervised approaches for dealing with Zero-day attacks (Ahmed *et al.*, 2016).

2.4 Conclusions and State of the Art

Summarizing the literature, the current “state of the art” in intrusion detection has moved far away from traditional firewalls and signature-based DPI systems. The cutting edge focuses on unsupervised behavioral analysis, utilizing concepts from Information Theory and Stochastic Process Modeling on traffic metadata. However, the common denominator of all research is that no single statistical feature or single sensor is sufficient. Traffic is excessively noisy, easily leading to False Positive or False Negative results. Consequently, industry evolution mandates the use of Multi-Sensor Fusion architecture, which synthesizes multiple features (volumes, frequencies, speeds) and cross-references results to create the most robust and reliable shield against unknown threats.

2.5 Comparative Method Analysis

Taking modern literature into account, this work adopts a hybrid approach. While our architecture is founded on proven state-of-the-art practices, such as the use of Shannon Entropy and 1st order stochastic models to avoid overtraining, it simultaneously differentiates itself by introducing innovative data engineering features. A typical example is the handling of discrete control marks (TCP Flags), which instead of being treated as individual texts or independent variables, are algorithmically grouped via binary compression (bitmasking) into a single integer, creating

64 unique states (see Code Snippet 1 in Appendix A). In combination with the Multi-Sensor Fusion architecture that covers the “blind spots” of individual sensors, and the use of specialized mathematical transformations such as the logarithmic scale:

$$\log p(x) = \log(1 + x) \quad (2)$$

to avoid computational collapses (see Code Snippet 2 in Appendix A), the proposed system expands the capabilities of the existing state of the art. All these architectural choices are analyzed extensively in the following methodology chapters.

2.6 Deviation Evaluation (Cross-Entropy)

During the detection phase, the system monitors current traffic and calculates the deviation from the training model via Cross-Entropy Loss (*Zervas, 2025*). The operation is based on the reduction of probabilities into information values.

However, a serious mathematical issue arises when the system encounters a transition during testing that did not exist anywhere in the training set (i.e., $p = 0$). Because the logarithm of zero is undefined and tends to minus infinity, the Laplace smoothing technique is used (*Ye et al., 2004*) (see Code Snippet 4 in Appendix A). For every unknown transition, we assign an extremely small probability of the order of 1.0×10^{-9} . The negative base-2 logarithm for 10^{-9} equals approximately 29.9 bits. This number skyrockets the deviation to outrageous levels, providing an indisputable alarm signal for the existence of highly anomalous behavior.

3 Methodology

The system architecture is divided into three distinct phases: data processing, statistical model training (Training), and the production of the detection system (Detection).

The training is based on the philosophy of Unsupervised Anomaly Detection. We train the system exclusively on absolutely healthy traffic, allowing it to learn the “normal pulse” of the network without any knowledge of the types of attacks it will be called upon to face.

3.1 Dataset Description

For the evaluation and training of the system, a modern and extremely extensive network traffic dataset was selected, capable of simulating realistic load and attack conditions. A good practice of this methodology concerns how this dataset is managed and integrated into the detection system. Unlike conventional academic approaches that rely on local downloading and storage of static files on a hard drive, the proposed Data Pipeline was designed to draw the volume of data dynamically and directly from the AWS Open Data Registry. This choice makes the system fully autonomous and immediately reproducible by any third-party researcher, eliminating local storage limitations. At the same time, it makes the architecture by definition ready (Cloud-native) for immediate deployment and scaling in cloud computing infrastructures, where the flow of network data is continuous.

However, the utilization of raw network data on such a scale requires an extremely strict preprocessing stage before the data is fed into the Markov chain sensors. For this reason, a robust processing chain (the well-known Data Pipeline) was

created, the first phase of which was dedicated to **“aggressive” cleaning** and filtering. Initially, a scan of the file dimensions was performed, from which numerous **“dead” columns** that contained absolutely no information or were filled with undefined values (NaN/Null) were entirely identified and removed, unnecessarily reducing computational speed. Additionally, strict row dropping rules were applied: packets with missing flow characteristics (missing values), corrupted records, or duplicates resulting from logging tool errors were deleted. The result of this intensive filtering was the production of a compact, “cleaned,” and reliable set of observations, which constitutes the ideal substrate for the unbiased training of Information Theory algorithms, guaranteeing that any deviations detected later will be clearly due to behavioral anomalies and not data noise.

3.2 Feature Selection and Data Engineering

From the multitude of available NetFlow features, five basic ones were selected, inspired by evaluation studies in information security (*Lee and Xiang, 2001*):

- **Time Duration (Flow_Duration):** Critical for detecting slow attacks (e.g., Slowloris) that try to keep connections open for huge periods of time.
- **Dispersion Degree (Out_Degree):** Represents the number of different destinations with which a computer communicates. Changes in this constitute the primary indication for Port Scanning attacks.
- **Total Number of Packets (Tot_Fwd_Pkts):** Reflects the existence of Brute-Force attacks where a massive number of requests are sent in a short period of time.
- **Total Packet Volume (TotLen_Fwd_Pkts):** Serves as the main detection indicator for massive data extraction or Data Exfiltration attempts.
- **Control Marks ‘TCP_Flags_Int’:** The six distinct control marks (SYN, ACK, FIN, RST, PSH, URG) were compressed into the binary system (bitmasking), so the mark is not just text, but produces exactly one integer from 0 to 63. This creates exactly **64 unique and non-overlapping states**, ideally integrating the different flags into a single mathematical variable (see Code Snippet 1 in Appendix A).

For the rapid processing of these files, which exceeded several Gigabytes, we replaced traditional single-thread tools with the Polars library, written in Rust, which offers multi-threaded architecture and vectorized command execution.

3.3 Discretization and Smoothing

The mathematics of entropy require discrete states. However, network data (like time or size) is continuous (except for the control marks we just mentioned), so discretization of these other variables had to be performed.

The Equal-Width Binning method was used, which separated each continuous variable into exactly 8 categories. The problem with extreme network numbers is that they create huge, empty areas. To protect the distribution, all numbers were converted to a logarithmic scale before categorization. Specifically, to avoid mathematical errors when a feature (like ‘Out_Degree’) takes the value zero (where $\log(0)$ is undefined), we used the function $\log(1 + x)$ or $\log 1p(x)$, which ensures that $\log 1p(0) = 0$ (see Code Snippet 2 in Appendix A).

3.4 Model Architecture and "Training" Process

The heart of the detection system was developed in Python, utilizing specialized data analysis libraries for the rapid processing of discretized flows. The training architecture was designed with feature independence in mind: the model does not treat traffic as a single, inseparable whole, but is trained autonomously for each individual network feature (Flow Duration, TCP Flags, etc.).

During the "training" process, the algorithm scans observations of absolutely normal traffic (Training Set). For each independent feature, it maps the sequence of states and creates an exclusive Transition Probability Matrix. This matrix mathematically captures the probability of the system transitioning from the current state to the immediately next one, thus establishing the rule of the 1st order Markov Chain (see Code Snippet 3 in Appendix A). Based on this matrix, the Baseline Shannon Entropy of the feature is then calculated, acting as the digital footprint of the network's "healthy" uncertainty.

Although literature suggests the use of 1st order, a strictly experimental approach was used in the methodology to verify this theory. The training algorithm was parameterized and executed multiple times, in order to construct transition matrices not only for $k = 1$, but also for higher Markov orders (such as $k = 2$, $k = 3$, and $k = 5$). The goal of this experimental expansion was to determine whether a "deeper memory" of the past improves the system's generalization ability or drives it to a mathematical collapse. Indeed, for order $k = 1$, the results are optimal. The thorough analysis of these baseline experiments is presented in Chapter 5.

The architecture concludes in the evaluation phase (Detection / Inference). Having now "learned" the distribution of healthy traffic, the system receives new, unknown data. Instead of recalculating their entropy from scratch, it uses the Cross-Entropy function. That is, it compares the observed sequence of the new data with the probabilities it has stored in the training transition matrix. Through this calculation, the system extracts a deviation value (Loss): the higher the value of the cross-entropy, the more certain the system is that the observed traffic does not belong to the normal network profile, thus sounding an alarm for a possible cyberattack.

4 Implementation

4.1 Development Environment

For the implementation of the system and the execution of experiments, the Google Colaboratory (Colab) environment was chosen, interfaced with Google Drive (google.colab.drive).

This specific choice was dictated by the massive volume of network data, the processing of which required dynamic resource allocation and available memory (RAM) that often exceeds the capabilities of a conventional computer.

Through Colab, the correct execution of the code, which was developed entirely in Python 3, was ensured.

Regarding the software, special emphasis was placed on optimizing data engineering, and for this reason, the traditional Pandas library was abandoned, and the Polars library was integrated. As mentioned earlier, Polars is ideal for fast processing of large data packets due to its multi-threaded operation, thereby dramatically reducing dataset filtering times.

It is worth noting that the Markov Chain training algorithm was not based on any ready-made machine learning library but was developed from scratch.

For counting the chain’s states and transitions, built-in Python structures (collections.defaultdict) were utilized, while the NumPy library, which guarantees fast vectorized calculations, was extensively used to construct the transition matrices, calculate probabilities, and **extract the Entropy**.

Additionally, the yaml and json libraries were used to manage system configuration and store the trained model.

Finally, in the evaluation and results visualization stage, the Plotly libraries were employed to produce interactive detection graphs, along with Matplotlib and Scikit-Learn for calculating evaluation metrics (Precision, Recall, F1-Score, ROC-AUC) and rendering Confusion Matrices.

4.2 Multi-Sensor Architecture

The problem with many anomaly detection systems is that they rely on a single metric (one sensor) to draw conclusions. As has been proven in literature, using only one sensor inevitably leads to False Negatives (*Nychis et al., 2008*).

For example, a smart, distributed DDoS attack might use hundreds of different IP addresses, resulting in the ‘Out_Degree’ feature looking completely normal. If the system only looked at this, the attack would pass unnoticed.

For this reason, the multi-sensor fusion architecture was developed. The simultaneous use of all five features as independent observers eliminates this problem. The system analyzes the Markov chains of all features simultaneously and acts as a logical OR gate (see Code Snippet 5 in Appendix A). If the normalized deviation of even a single sensor exceeds the global threshold (> 1.0), an alarm is declared, providing comprehensive coverage across all attack surfaces.

5 Experiments and Results

Our experimental analysis confirms the presented theory. By examining the results during model testing, we demonstrated the necessity of using order $k = 1$.

5.1 Baseline Results

Table 1 presents the number of unique states detected by the system in a sample of hundreds of thousands of packets, and how the baseline entropy collapses as the order increases.

Table 1: *Number of Unique States and Baseline Entropy Depending on Model Order k (Sequence Size: 488,381)*

Feature	$k = 1$		$k = 2$		$k = 3$		$k = 5$	
	States	H	States	H	States	H	States	H
TCP_Flags_Int_sym	11	2.035	97	1.872	656	1.639	13657	1.119
Flow_Duration_sym	7	2.534	49	2.533	343	2.514	15090	1.832
TotLen_Fwd_Pkts_sym	8	1.444	-	-	-	-	2076	1.318
Out_Degree_sym	1	0.000	-	-	-	-	1	0.000

The table above captures with absolute mathematical clarity the impact that increasing the Markov Chain's order k has on the model's complexity and generalization capability.

Analyzing the behavior of individual features, three main conclusions emerge that justify the final architectural choice of $k = 1$.

First, the phenomenon of **State-Space Exponential Explosion** is observed. For highly variable features, such as TCP Flags and Flow Duration, the 1st order ($k = 1$) produces a compact and normal number of states (11 and 7 respectively), which is absolutely sufficient to map the fundamental communication rules (e.g., the agreement rules of the TCP Handshake).

Conversely, forcing the system to memorize sequences of five consecutive packets ($k = 5$) causes the possible states to skyrocket to 13,657 and 15,090.

This dramatic increase proves that the model passes from the rule-learning phase to the phase of **sterile memorization**.

The simultaneous drop in Entropy (e.g., in TCP Flags from 2.035 to 1.119) confirms that the transition matrix fills with absolute probabilities of the order of 100% or 0%, rendering the model incapable of recognizing any normal -but slightly varied- future traffic, which would lead to a rapid increase in False Positive alarms.

Second, the behavior of the Out Degree feature confirms the stability of the network topology during normal operation.

The fact that this feature presents only 1 state with zero entropy ($H=0.000$) in the 1st order means that network dispersion is absolutely stable and deterministic.

Consequently, it is mathematically impossible for an increase in order to produce new information (since a sequence of 5 identical values remains a single unique state).

This explains why Out Degree maintains 1 state and zero entropy even at $k = 5$.

Third, for variables like the Total Length of Forward Packets, the trend of exponential growth (from 8 states in $k = 1$, to 2,076 in $k = 5$) is so rapid and clear that the computational extraction of intermediate orders ($k = 2, 3$) was deemed meaningless and omitted (as indicated by the dashes in the table).

In conclusion, the 1st order Markov Chain proves to be the "golden ratio," as it manages to maintain the system's uncertainty at levels that allow for healthy traffic variability, while simultaneously keeping computational costs to a minimum.

5.2 Graphical Evaluation

Entropy Drop Depending on Order k

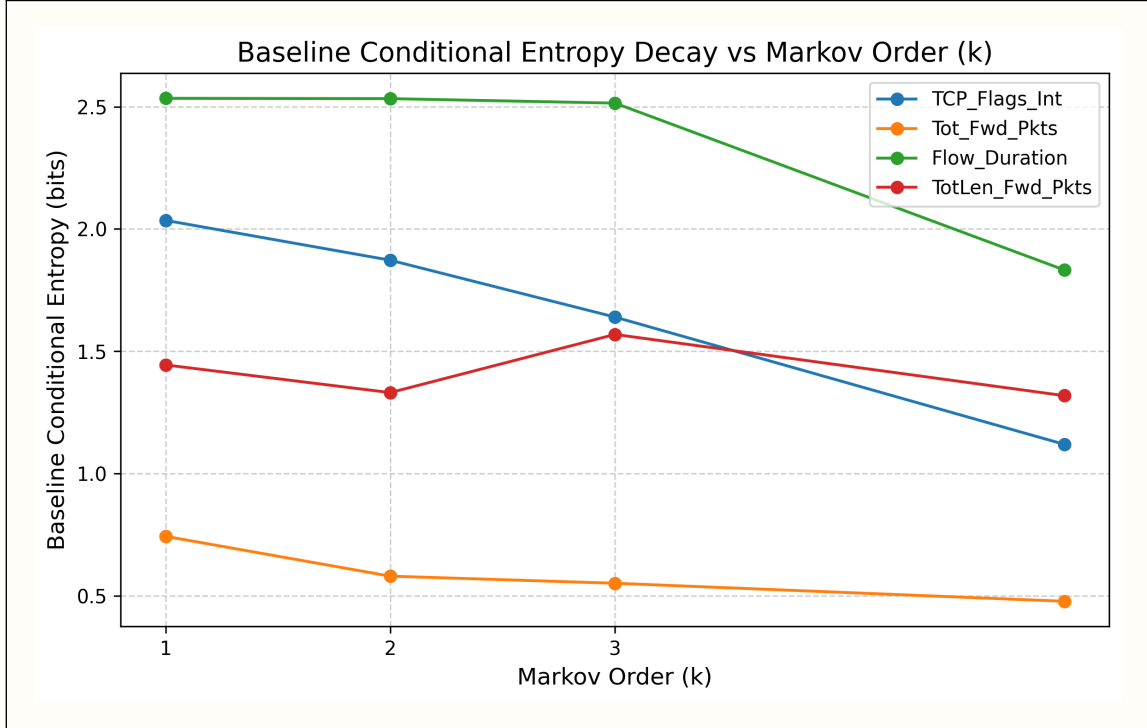


Figure 1: Drop in entropy as the model order (k) increases

The figure above visually confirms the fundamental rule of Information Theory: as the amount of available information from the past increases (i.e., order k), the uncertainty for the next state — and consequently the Conditional Entropy — decreases.

However, the behavior of this drop differs radically depending on the nature of each feature.

For example, the entropy of TCP Flags (blue line) presents a steady and almost linear drop, indicating that the system progressively “memorizes” the protocol’s control sequences.

On the other hand, Flow Duration (green line) demonstrates strong resilience up to order $k = 3$, keeping its entropy high, before collapsing abruptly at $k = 5$, a point at which the state space becomes excessively sparse.

The behavior of the total packet length (red line) is interesting, as it displays a slight fluctuation (drop at $k = 2$, rise at $k = 3$) before its final drop.

This non-monotonic behavior indicates artificial anomalies created when a limited number of observations are forcibly distributed across exponentially increasing available states.

Conversely, features with inherently low variance, such as the total forward packets (orange line), start from already low entropy levels, proving that they are highly deterministic by nature, rendering higher orders practically redundant.

Multi-Sensor Behavior Analysis

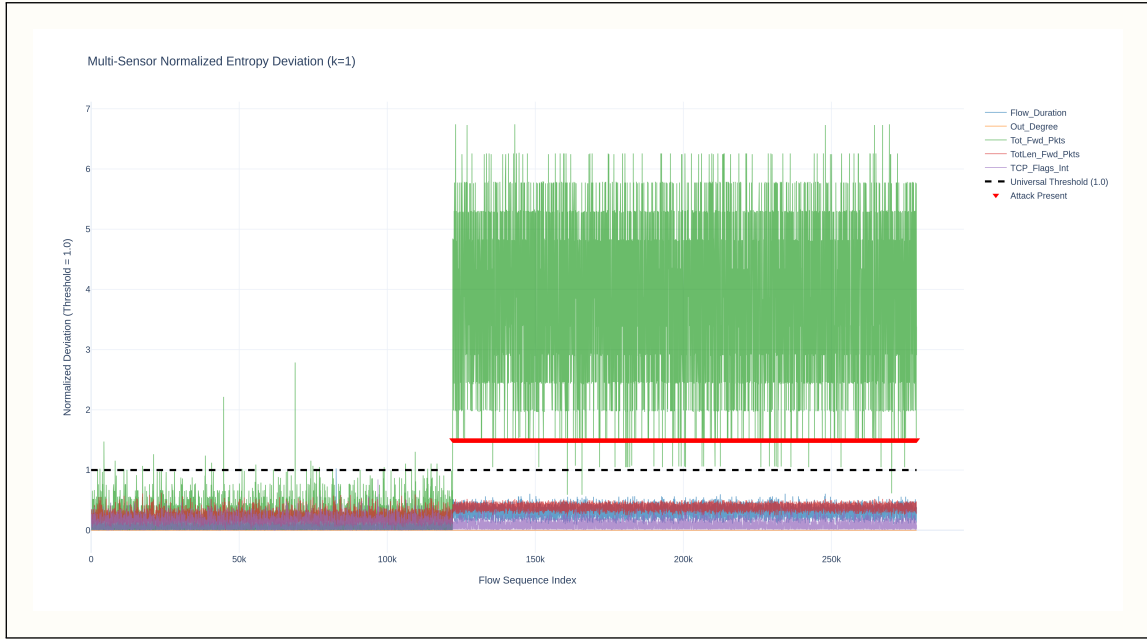


Figure 2: *Normalized Deviation of multiple sensors during attacks*

The figure above represents the culmination of the system’s experimental evaluation, as it clearly visualizes the superiority of the Multi-Sensor architecture under real attack conditions. The horizontal axis depicts the temporal sequence of network traffic (Flow Sequence Index), while the vertical axis measures the Normalized Deviation from the normal Markov profile.

The dashed black line defines the global tolerance threshold (Threshold=1.0), above which the system sounds an alarm.

Simultaneously, the red indicator line on the graph shows the actual time duration of the attacks (Ground Truth).

The most critical observation concerns the differentiated reaction of the sensors at the onset of the attack.

As can be seen, certain features (such as the blue and purple lines) remain “calm” below the alarm threshold even during the attack.

If the detection system relied exclusively on these individual sensors, it would fail miserably, recording a critical False Negative result.

However, thanks to the simultaneous monitoring of multiple dimensions, the sensor monitoring forwarded packets (green line) reacts violently, with its deviation skyrocketing to values above 6.0, emphatically piercing the 1.0 threshold exactly at the moment the red indicator of the attack begins.

The operation of the sensors as a **“Logical OR Gate”** (see Code Snippet 5 in Appendix A) ensures that the detection of the anomaly by even a single sensor is enough to protect the entire network, practically proving the reliability and robustness of the proposed methodology against “invisible” threats.

Detection Performance Evaluation

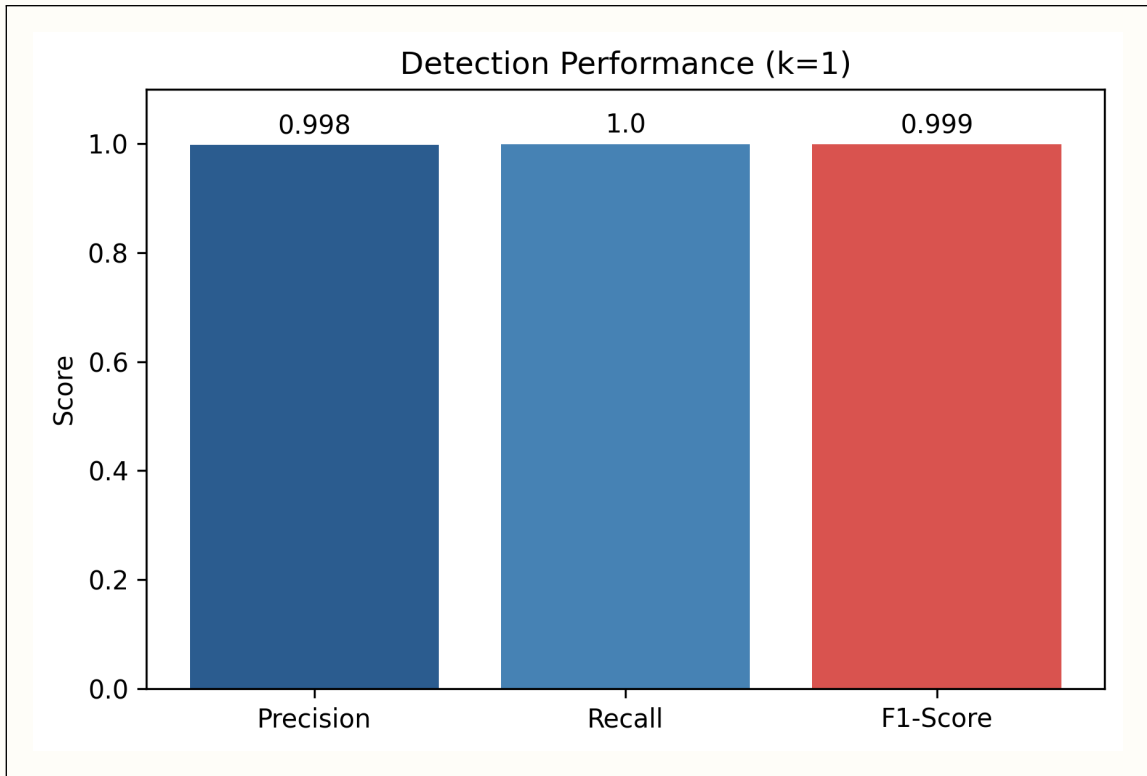


Figure 3: Performance evaluation (ROC Curves & Confusion Matrix) of the model

As depicted in the graph above, the system achieves almost perfect results, with Precision reaching 0.998, Recall an absolute 1.0, and F1-Score at 0.999.

At this point, the question reasonably arises whether these exceptionally high results are indicative of overtraining or reflect a false reality ("too good to be true"). The answer lies within the core of the architecture itself, which makes **overtraining structurally impossible**.

First, the methodology is strictly unsupervised.

The system was never trained on malicious data, so it is impossible for it to have memorized attack signatures.

Second, as proven by the state-space analysis (Table 1), the deliberate choice of 1st order ($k = 1$) acted as a natural switch, preventing the system from memorizing specific, noisy packet sequences that would lead to excessive compliance.

The absolute 1.0 in Recall (zero False Negatives) is not due to the ability of a single, over-trained algorithm, but to the Multi-Sensor Fusion architecture.

The system operates as a multidimensional net: if an attacker tries to mimic normal traffic in the Flow Duration feature to avoid detection, they will inevitably cause a severe anomaly in some other feature (like Total Forward Packets), triggering the corresponding sensor.

Correspondingly, the impressive Precision (0.998) stems from $k = 1$'s ability to maintain sufficient uncertainty to forgive the normal -even if rare- fluctuations of healthy traffic, keeping False Positives at practically zero levels.

6 Discussion

6.1 Interpretation of Results

Although the experimental results (F1-Score = 0.999) prove the exceptional ability of the Multi-Sensor Fusion architecture to distinguish malicious from healthy traffic, their correct interpretation requires acknowledging the **inherent biases** of the system.

The high performance does not imply the discovery of a "universal" detection rule but indicates that the model has perfectly adapted (environment bias) to the topology and habits of the specific network on which it was trained.

In other words, the transition matrices of the Markov Chain are inextricably linked to the profile of each organization.

If the same -already trained- model were applied to a different corporate network, its performance would collapse, as new healthy behaviors would be incorrectly categorized as anomalies.

Therefore, the proposed system does not offer a plug-and-play attack filter, but a highly efficient self-learning methodology, which requires **local training** for each new infrastructure.

Additionally, the system carries the classic theoretical bias of anomaly detection: the arbitrary assumption that "every statistical deviation equals an attack".

In realistic conditions, extreme but legitimate events, such as a massive software update or an unexpected traffic spike due to remote work, would cause a significant deviation in entropy.

Although the multi-sensor design largely mitigates this phenomenon, the system naturally remains "biased" in favor of security, preferring to sound a False Positive alarm over allowing a potential threat.

6.2 System Limitations

Despite its success, the system presents four critical limitations worth noting:

1. **Absence of Deep Inspection:** It is more correct to provide a collaborative service between metadata analysis and DPI. In this case, the system exclusively analyzes traffic metadata, so attacks occurring at the application layer and containing malicious code without affecting volume or temporal behavior are completely invisible to the system.
2. **Low-and-Slow Attacks (APT):** Using $k = 1$, the system does not have 'deep memory' to catch attacks that evolve stealthily and slowly over days, with the inevitable consequence that the fluctuation is absorbed as "normal".
3. **Static and Offline Analysis:** The current system operates as a post-mortem tool that analyzes logs. To operate live (in real-time), it must be interfaced with packet capture tools like libpcap.
4. **Concept Drift:** Network behavior changes from network to network as it is also temporally variable and dependent. The static state transition matrix created today will become obsolete tomorrow, requiring continuous training or model updates to remain relevant in the future.

The choice of these limitations was however **conscious** in order to guarantee absolute explainability. It is worth noting that the system's architecture is fully extensible and can integrate additional sensors in the future, such as DPI analysis

or User Behavior Analytics, without altering the basic philosophy of the 1st order Markov Chain.

6.3 Additional Features and Extensions (BETA)

In order to make the detection system fully accessible in real conditions (beyond the level of raw code), an experimental Web application (Dashboard) based on the Streamlit library was additionally developed within the scope of this project. This graphical environment allows security analysts to immediately visualize the final performance metrics, inspect the interactive entropy deviation graphs on the Timeline, and perform "live" mathematical proofs of Information Theory on the dataset.

One of the most interesting extensions integrated into the Dashboard is the practical application of **Huffman Coding** as a secondary, independent indicator of attack detection, based on Shannon's Source Coding Theorem.

In a state of normal operation, network traffic is characterized by diversity and high entropy.

Consequently, the Huffman algorithm achieves a moderate compression ratio (e.g., an average of 2.71 bits per symbol).

Conversely, during an attack (e.g., DDoS), the network is flooded by millions of repetitive, predictable, and structurally identical packets. This excessive predictability flattens the source's entropy.

When the Huffman algorithm attempts to compress the attack traffic, the compression ratio increases dramatically, and the average bits per symbol drops unusually low (e.g., to 2.30).

This extremely high compression rate produces a boolean indicator (or an alarm threshold) that mathematically certifies -and from a completely different perspective- the existence of a malicious routine on the network, further enhancing the reliability of the Markov Chains.

7 Conclusions

7.1 Project Summary

The use of first-order Markov chains and Shannon entropy provides a reliable, lightweight, and fully explainable detection method. The Multi-Sensor architecture minimizes False Negatives and False Positives, while simultaneously combating the weaknesses of each sensor individually.

In an era where Deep Packet Inspection is becoming difficult due to encryption, behavior monitoring based on Information Theory, despite the limitations it carries against attacks of immense duration, constitutes a stable shield of protection. The detection of deviations in the data exchange pattern opens huge avenues for future live implementations.

7.2 Future Work

Although the current implementation proves the validity of the proposed methodology in a controlled environment and static datasets (offline datasets), the ultimate goal of the research is oriented towards the development of a real-time detection system (real-time NIDS).

This transition requires the replacement of batch processing with continuous data streaming mechanisms, where the system will monitor network traffic live.

In such a real-network simulation scenario, the calculation of entropy and the evaluation of Markov Chain transitions must be executed within strict sub-millisecond time windows, in order to ensure that monitoring will not introduce latency into the topology.

Additionally, the design of the Multi-Sensor Fusion has been deliberately structured with high extensibility, leaving fertile ground for future experimentation with a multitude of new variables.

In the next stages of implementation, the aim is to integrate additional sensory metrics, which will draw information either from deeper layers of the OSI model (where encryption allows), or from temporal deviations of multiple protocols.

The addition of new "sensors" in the live environment is expected to further thicken the theoretical protective "net" of the system, reducing escape margins for smart, polymorphic, or Zero-Day attacks, which might attempt to successfully mimic the behavior of healthy traffic on one or two individual axes.

References

- Ahmed, M., Mahmood, A. N., and Hu, J. (2016). A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60:19–31.
- Callegari, C., Giordano, S., and Pagano, M. (2009). On the use of compression algorithms for network anomaly detection. In *2009 IEEE International Conference on Communications*.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep learning*. MIT press.
- Lee, W. and Xiang, D. (2001). Information-theoretic measures for anomaly detection. In *Proceedings of the 2001 IEEE Symposium on Security and Privacy*, pages 130–143.
- Nychis, G., Sekar, V., Andersen, D. G., Kim, H., and Zhang, H. (2008). An empirical evaluation of entropy-based traffic anomaly detection. In *Proceedings of the 8th ACM SIGCOMM Conference on Internet Measurement*, pages 151–156.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.
- Ting, C., Field, R., Fisher, A., and Bauer, T. (2018). Compression analytics for classification and anomaly detection within network communication. *IEEE Transactions on Information Forensics and Security*.
- Ye, N., Ehiabor, T., and Zhang, Y. (2002). First-order versus high-order stochastic models for computer intrusion detection. *Quality and Reliability Engineering International*, 18(1):17–22.
- Ye, N., Zhang, Y., and Borrer, C. M. (2004). Robustness of the markov-chain model for cyber-attack detection. *IEEE Transactions on Reliability*, 53(1):116–121.
- Zervas, P. (2025). Βασικές Αρχές Θεωρίας Πληροφοριών. Διάλεξη 3.
- Zervas, P. (2026). Διακριτή Πηγή Πληροφορίας με Μνήμη. Διάλεξη 5.

A Appendix A: Code Snippets

This appendix presents the most important code snippets that implement the basic algorithms of the detection system. The full code can be found in the corresponding Jupyter Notebooks of the project.

A.1 - TCP Flags Bitwise Encoding

The following code synthesizes the six distinct TCP control flags (SYN, ACK, PSH, RST, URG, FIN) into a unique integer via a bitmask, producing exactly 64 unique states (0-63).

```
1 # Synthesize 'TCP_Flags_Int' from individual flag columns
2 # Formula: SYN + 2*ACK + 4*PSH + 8*RST + 16*URG + 32*FIN
3 flag_cols = {
4     "SYN": "SYN Flag Cnt",
5     "ACK": "ACK Flag Cnt",
6     "PSH": "PSH Flag Cnt",
7     "RST": "RST Flag Cnt",
8     "URG": "URG Flag Cnt",
9     "FIN": "FIN Flag Cnt"
10 }
11
12 df = df.with_columns(
13     (
14         pl.col(flag_cols["SYN"]) * 1 +
15         pl.col(flag_cols["ACK"]) * 2 +
16         pl.col(flag_cols["PSH"]) * 4 +
17         pl.col(flag_cols["RST"]) * 8 +
18         pl.col(flag_cols["URG"]) * 16 +
19         pl.col(flag_cols["FIN"]) * 32
20     ).alias("TCP_Flags_Int")
21 )
```

Listing 1: TCP Flags Bitwise Encoding (*data_pipeline.ipynb*)

A.2 - Log-Scale Binning of Continuous Variables

To convert continuous features into discrete states, the logarithmic transformation $\log_1 p(x) = \log(1 + x)$ is first applied to smooth extreme values, followed by Equal-Width Binning into 8 categories.

```
1 BINS = 8
2 numeric_cols = ["Flow_Duration", "Out_Degree",
3                 "Tot_Fwd_Pkts", "TotLen_Fwd_Pkts"]
4
5 # Compress extreme outliers using log transform
6 df_selected = df_selected.with_columns(
7     [pl.col(c).clip(lower_bound=0).log1p().alias(f"{c}_log")
8      for c in numeric_cols]
9 )
10
11 # Perform equal-width binning on the logged values
12 for col_name in numeric_cols:
13     col_log = f"{col_name}_log"
14     col_min = df_selected.select(pl.col(col_log).min()).item()
15     col_max = df_selected.select(pl.col(col_log).max()).item()
16
17     if col_max == col_min:
18         df_selected = df_selected.with_columns(
```

```

19         pl.lit(0).alias(f"{col_name}_sym")
20     )
21     else:
22         df_selected = df_selected.with_columns(
23             ((pl.col(col_log) - col_min) /
24              (col_max - col_min) * (BINS - 1))
25             .cast(pl.Int32)
26             .alias(f"{col_name}_sym")
27         )
28
29     # TCP Flags are already discrete (64 states)
30     df_selected = df_selected.with_columns(
31         pl.col("TCP_Flags_Int").cast(pl.Int32)
32         .alias("TCP_Flags_Int_sym")
33     )

```

Listing 2: Log-Scale Discretization (*data_pipeline.ipynb*)

A.3 - Markov Chain Transition Matrix Construction

The core of the training algorithm: it scans a sequence of symbols, counts the transitions between consecutive states, and constructs the Transition Probability Matrix for any order k .

```

1  from collections import defaultdict
2
3  SMOOTHING = 1e-9
4
5  def build_transition_matrix(sequence, k=1):
6      """Build a k-th order Markov chain transition
7      probability matrix from a symbol sequence."""
8      counts = defaultdict(lambda: defaultdict(int))
9      for i in range(len(sequence) - k):
10         state = str(tuple(sequence[i:i+k]))
11         nxt = str(sequence[i+k])
12         counts[state][nxt] += 1
13
14     prob = {}
15     for state, nexts in counts.items():
16         total = sum(nexts.values())
17         prob[state] = {s: c / total for s, c in nexts.items()}
18     return prob

```

Listing 3: Markov Chain Transition Matrix Builder (*training.ipynb*)

A.4 - Cross-Entropy Loss Calculation

The detection function: given a new sequence and the trained transition matrix, it calculates the average cross-entropy. When a transition does not exist in the training model, it is replaced with Laplace smoothing ($p = 10^{-9}$), skyrocketing the deviation.

```

1  def window_cross_entropy(sequence, baseline_tm,
2                          k=1, smoothing=1e-9):
3      """Calculate the cross-entropy of a sequence given
4      the baseline transition matrix from training."""
5      if len(sequence) <= k:
6          return 0.0
7
8      log_prob_sum = 0.0

```

```

9     for i in range(len(sequence) - k):
10         state = str(tuple(sequence[i:i+k]))
11         nxt = str(sequence[i+k])
12
13         # Get probability from baseline, else smoothing
14         if state in baseline_tm \
15             and nxt in baseline_tm[state]:
16             p = baseline_tm[state][nxt]
17         else:
18             p = smoothing # Laplace: ~29.9 bits penalty
19
20         log_prob_sum += -np.log2(p)
21
22     return log_prob_sum / (len(sequence) - k)

```

Listing 4: Cross-Entropy Loss Calculator (*detection.ipynb*)

A.5 - Multi-Sensor Fusion Detection Loop

The central evaluation loop: it scans the test traffic in windows, calculates the normalized deviation of each sensor, and sounds an alarm if even one sensor exceeds the global threshold (OR logic gate).

```

1  for i in range(0, len(label_full) - window_size + 1,
2                 stride):
3      is_alert = False
4      max_norm_dev = 0.0
5
6      for f in features:
7          seq_window = seqs[f][i : i + window_size]
8          ce = window_cross_entropy(
9              seq_window, models[f],
10             k=K_ORDER, smoothing=SMOOTHING
11         )
12         dev = abs(ce - base_H[f])
13
14         # Normalize: > 1.0 means threshold crossed
15         norm_dev = dev / thresh[f] if thresh[f] > 0 else 0
16         feature_deviations[f].append(norm_dev)
17
18         if norm_dev > 1.0:
19             is_alert = True # OR-gate logic
20         if norm_dev > max_norm_dev:
21             max_norm_dev = norm_dev
22
23     final_alerts.append(1 if is_alert else 0)

```

Listing 5: Multi-Sensor Fusion Detection Loop (*detection.ipynb*)